FPC Mobile — Evidencia del Sprint 3

Base de datos, índices, trigger de estado y políticas RLS, con su verificación

Proyecto	FPC Mobile — registro y trazabilidad de cilindros de gas para FPC Servicios
Documento	Evidencia del Sprint 3. Qué se construyó, cómo se verificó y qué quedó fuera, con los resultados medidos contra el proyecto real de Supabase
Equipo	Bastián Núñez (app móvil y UI) · Nicolás Castillo (base de datos y backend) · Nicolás Cifuentes (análisis, documentación y pruebas)
Autor	Nicolás Castillo — Ingeniería en Informática, Duoc UC, sede San Joaquín
Fecha	23 de octubre de 2026

1. Qué cubre el Sprint 3

Cinco tarjetas del tablero: crear el esquema, el índice único de `uuid_local`, el trigger de estado del cilindro, las políticas RLS por rol y la documentación del modelo.

Plazo original: 28 de septiembre al 11 de octubre. Terminado el 23 de octubre, con doce días de atraso. La causa está documentada: el sprint estuvo bloqueado esperando el maestro de cilindros del cliente, que llegó el 21 de octubre. El esquema se escribió y verificó en los dos días siguientes.

2. Qué se construyó

Seis migraciones, versionadas en el repositorio bajo `supabase/migrations/`:

MIGRACIÓN	QUÉ CREA
`esquema_inicial`	6 dominios enumerados, 7 tablas, 2 triggers de servicio, RLS habilitado en las siete
`indices`	10 índices explícitos (25 en total contando los implícitos de clave primaria y única)
`trigger_estado_cilindro`	El recálculo del estado derivado del envase, en schema `private`
`politicas_rls`	`mi_rol()`, 29 políticas y los `GRANT` por rol
`carga_catalogo_y_parque`	77 productos y 2.178 cilindros reales
`revocar_rls_auto_enable`	Cierra el hallazgo del advisor de seguridad

Las siete tablas son `usuarios`, `camiones`, `clientes`, `sucursales`, `productos`, `cilindros` y `movimientos`. El detalle campo por campo está en el diccionario de datos versión 2.

> **Tres decisiones de diseño que conviene poder defender**
>
> **El historial no se puede borrar.** `movimientos` no recibe política de `DELETE` para ningún rol, ni siquiera administrador. Sin política, RLS lo niega a todos. En un sistema que controla carga peligrosa, un historial borrable no sirve como respaldo; las correcciones son `UPDATE` del

> administrador y quedan registradas.
>
> **El trigger recalcula en vez de aplicar.** Lee el movimiento más reciente del envase en lugar de aplicar el que acaba de llegar. Así un movimiento que sincroniza tres días tarde no pisa el estado correcto, y además el estado queda bien cuando un administrador corrige un movimiento viejo.
>
> **Las funciones internas viven en un schema `private` sin permiso de uso para la aplicación.** Las políticas pueden llamarlas, pero el código de la app no. Esto funciona porque PostgreSQL evalúa las expresiones de las políticas con los privilegios del dueño de la tabla.

3. Verificación de estructura y datos

Medido contra el proyecto real de Supabase después de aplicar las seis migraciones.

COMPROBACIÓN	ESPERADO	OBTENIDO
Tablas en `public`	7	7
Índices	25	25
Políticas RLS	29	29
Tablas **sin** RLS habilitado	0	0
Productos en el catálogo	77	77
Cilindros en el parque	2.178	2.178
Cilindros propiedad del cliente	89	89
Productos sin capacidad (solo PRO)	1	1
Cilindros fuera de planta	0	0
Claves foráneas rotas	0	0
Números de serie repetidos	0	0

Advisor de seguridad de Supabase: sin hallazgos. Antes de la sexta migración reportaba dos avisos sobre `public.rls_auto_enable()`, una función de la plataforma ejecutable desde la API REST por `anon` y `authenticated`. Tras revocar ese permiso, el advisor quedó limpio y se comprobó directamente que esos roles ya no pueden ejecutarla.

4. Pruebas de comportamiento

Diez comprobaciones, todas aprobadas. Se ejecutaron **dentro de una transacción que se revierte al final**, de modo que no quedó ningún dato de prueba en la base. La salida completa está en `supabase/PRUEBAS_ESQUEMA.md`, versionada en el repositorio.

4.1 El movimiento que llega tarde

Es la razón de ser del trigger: la app funciona sin conexión y un movimiento puede sincronizar tres días después, llegando al servidor **después** de otro que ocurrió más tarde en terreno.

PASO	RESULTADO
`entrega` con fecha de hoy	El envase queda `en_cliente`, con sucursal y fecha de hoy ■

PASO	RESULTADO
`carga` con fecha de **hace tres días**	**El envase no cambia** ■
`retiro` con fecha de ahora	Queda `en_camion`, con los derivados en nulo ■

Si el segundo paso hubiera cambiado el estado, el trigger estaría mal y el panel mostraría cilindros en el lugar equivocado.

4.2 Idempotencia de la sincronización

Reenviar el mismo movimiento no duplica: el segundo `insert` con el mismo `uuid_local` afectó **0 filas**, quedó **1** registro y el estado del envase no cambió. Es lo que permite que el teléfono reintente sin miedo cuando la conexión se corta a medias.

4.3 Permisos por rol

PRUEBA	RESULTADO
Técnico inserta a nombre de **otro** usuario	Rechazado, error 42501 ■
Técnico inserta a nombre propio	1 fila ■
Técnico edita su propio movimiento	0 filas afectadas, el valor no cambió ■
Técnico A ve sus movimientos	5 ■
Técnico B ve los del técnico A	0 ■

> **Un detalle que casi produce un falso positivo**
>
> Un `UPDATE` o `DELETE` bloqueado por RLS **no lanza excepción**: simplemente afecta cero filas, porque la cláusula `using` deja la fila fuera del conjunto. Una prueba que solo verifique «no hubo error» da por aprobado algo que no probó. Las pruebas de este sprint miden **filas afectadas**.
>
> Esto importa más allá del test: **la plataforma web del Sprint 6 no puede asumir que un `update` sin error tuvo efecto.**

5. Qué quedó fuera, y por qué

No se corrió `supabase db reset`: no hay Docker en el equipo de desarrollo. Por lo tanto no hay evidencia, desde la propia herramienta, de que las migraciones reconstruyan el esquema desde una base vacía.

Existe **evidencia equivalente pero no idéntica**: durante su desarrollo, las cinco primeras migraciones se aplicaron sobre un **PostgreSQL 16 vacío** y levantaron el esquema completo —7 tablas, 6 dominios, 25 índices, 29 políticas y los 2.178 cilindros— en 0,7 segundos. La sexta se probó por separado contra una base donde la función de plataforma no existe, para confirmar que su bloque condicional no rompe un reset.

Las pruebas de RLS simulan la sesión con claims JWT dentro de la base. Falta probarlas con un inicio de sesión real desde la aplicación, lo que corresponde al Sprint 5.

Las pruebas no dejaron rastro en la base por diseño, al correr en una transacción revertida. La evidencia es el documento de salida versionado, no datos residuales.

6. Correcciones pendientes en el tablero

- **«Crear esquema Supabase (cinco tablas)»**: el esquema crea **siete**. Corregir el título.
- **«Índice único uuid_local en movimientos»**: la migración **no crea** ese índice, porque

`uuid_local` es la clave primaria y PostgreSQL lo crea solo. La tarjeta se cumple, pero por una razón distinta de la que dice su nombre.

- El sprint cerró **doce días después** de su plazo y así debe aparecer en el informe.